

3.4 节名词解释与讲稿

面向 SSD 特性的训练 I/O 优化方法

这一节讨论的是：当大模型训练数据放不进 GPU 显存时，如何把一部分数据卸载到 CPU 内存和 SSD 中，并且让 SSD 的读写尽量高效、不要拖慢训练，同时减少无效写入，延长 SSD 使用寿命。

可以先用一句话理解本节：

大模型训练中会产生大量参数、梯度、优化器状态和中间激活值。GPU 显存不够时，需要把部分数据放到 CPU 内存和 SSD 上。本节研究如何根据 SSD 喜欢“大块、顺序、少随机、少覆盖”的特性，重新组织训练数据的读写方式。

一、神经网络训练基础名词解释

1. Tensor：张量

Tensor 可以理解为神经网络中的基本数据容器。
它本质上就是多维数组。

例如：

- 一个数字是 0 维 Tensor；
- 一个向量是 1 维 Tensor；
- 一张图片可以表示成 3 维 Tensor，例如 [高度，宽度，通道数]；
- 一批输入数据可以表示成 4 维 Tensor，例如 [batch size，通道数，高度，宽度]。

在大模型训练中，参数、梯度、activation、优化器状态，基本上都以 Tensor 的形式存在。

本节中提到“不要按 Tensor 粒度逐个写 SSD”，意思是：

如果每个 Tensor 都单独读写 SSD，请求会非常多，而且很多 Tensor 很小，会导致大量小 I/O，效率很低。

2. Layer：网络层

Layer 是神经网络中的计算单元。

例如在 Transformer 模型中，一个模型通常包含很多层，每一层会包含 attention、MLP、normalization 等模块。训练时，数据会按照 layer 顺序依次经过模型。

可以简单理解为：

输入数据 → 第 1 层 → 第 2 层 → 第 3 层 → ... → 输出结果

本节中经常提到“按照 layer 组织数据”，是因为训练过程的访问顺序往往和 layer 顺序有关。如果系统知道接下来要计算第 10 层，就可以提前把第 10 层需要的数据从 SSD 读到 CPU 或 GPU 中。

3. Parameter：模型参数

Parameter 是模型需要学习的变量，例如权重矩阵和偏置。

神经网络训练的目标就是不断调整这些参数，使模型输出结果越来越接近真实答案。

例如一个线性层可以写成：

输出 = 输入 × 权重 + 偏置

这里的“权重”和“偏置”就是 parameter。

在大模型中，parameter 的规模非常大，可能达到数十亿、数百亿甚至更多。GPU 显存无法完整保存全部参数时，就需要把部分参数分片存放到 CPU 内存或 SSD 中。

4. Parameter shard：参数分片

Shard 是“分片”的意思。

Parameter shard 就是把完整模型参数切成多个小块，每个 GPU、CPU 或 SSD 只保存其中一部分。

为什么要分片？

因为单个 GPU 放不下完整模型参数。
所以系统会把参数切开，例如：

完整参数 = shard 0 + shard 1 + shard 2 + shard 3

不同 rank 或不同设备保存不同 shard。
训练时，需要哪个 shard，就提前把它读取到计算设备中。

在 SSD 卸载训练中，parameter shard 是非常重要的数据对象，因为它直接参与 forward 和 backward。如果参数没有及时从 SSD 读出来，GPU 就会等待，训练速度会下降。

5. Forward：前向传播

Forward 是神经网络根据输入数据计算输出结果的过程。

例如：

输入文本 → 模型计算 → 输出预测结果

在 forward 阶段，模型会一层一层计算。每一层都会使用对应的参数，并产生中间结果。

这些中间结果就是 activation。

forward 阶段的特点是：

- 需要读取模型参数；
- 会产生 activation；
- 计算顺序通常是从前往后；
- 可以根据 layer 顺序提前预取后续层参数。

6. Activation：激活值 / 中间激活

Activation 是 forward 过程中产生的中间结果。

为什么 activation 重要？

因为 backward 阶段计算梯度时，需要用到 forward 阶段的一些中间结果。
也就是说，activation 在 forward 之后不能马上全部丢掉，否则 backward 就没法计算。

例如：

```
forward 产生 activation  
backward 需要 activation 来计算梯度
```

但是 activation 的数据量通常很大，尤其是在大 batch、长序列、大模型训练中。
如果全部保存在 GPU 显存里，会占用大量显存。

因此系统可以选择：

1. 保存在 GPU 显存；
2. 卸载到 CPU 内存；
3. 必要时 spill 到 SSD；
4. 不保存全部 activation，而是在 backward 时重新计算。

7. Activation spill：激活值溢出到 SSD

Spill 可以理解为“放不下时溢出到下一级存储”。

Activation spill 指的是：

当 GPU 显存或 CPU 内存不足以保存 activation 时，把部分 activation 临时写入 SSD。

但是要注意：

activation 写 SSD 通常不是首选策略。

更优先的策略一般是：

```
activation checkpointing / recomputation  
CPU offload  
必要时 SSD spill
```

原因是 activation 生命周期短，而且 SSD 比 GPU 显存和 CPU 内存慢。

如果大量 activation 频繁写 SSD，容易拖慢训练。

所以本节中提到 activation spill 时，应该理解为一种“内存压力较大时的补充策略”。

8. Backward：反向传播

Backward 是根据损失函数计算梯度的过程。

训练时，模型先 forward 得到预测结果，然后和真实答案比较，得到 loss。
接着 backward 根据 loss 反向计算每个参数应该如何调整。

可以简单理解为：

forward：用参数算结果

backward：根据错误计算每个参数的修改方向

backward 的特点是：

- 需要 forward 阶段保存的 activation；
- 会产生 gradient；
- 执行顺序通常和 forward 相反；
- 是训练关键路径的一部分。

9. Gradient：梯度

Gradient 表示参数应该如何更新。

如果 parameter 是模型当前的权重，那么 gradient 就告诉优化器：

这个参数应该增大还是减小，增大/减小多少

在 backward 阶段，每个 parameter 都会对应一个 gradient。

gradient 的生命周期通常比较短。

它在 backward 中产生，在 optimizer step 中被使用，更新完参数后就可以释放。

因此，在 SSD 卸载训练中，gradient 一般不应该长期写入 SSD。

只有在内存压力非常大时，才考虑短生命周期 spill。

10. Gradient shard：梯度分片

Gradient shard 是梯度的分片形式。

在分布式训练中，每个 GPU 或 rank 只保存部分梯度，或者在通信之后得到对应的梯度分片。

它的生命周期通常比 parameter 和 optimizer state 短。

大致过程是：

```
backward 产生 gradient
gradient 参与通信或聚合
optimizer 使用 gradient 更新参数
gradient 释放
```

所以它的 SSD 策略通常是：

尽量不要落盘；必要时只作为短生命周期临时数据处理。

11. Optimizer：优化器

Optimizer 是根据 gradient 更新 parameter 的算法。

常见优化器包括 SGD、Adam、AdamW 等。

简单理解：

```
parameter_new = parameter_old - 学习率 × gradient
```

不过像 Adam 这样的优化器不只使用当前 gradient，还会维护额外状态，例如：

- 一阶矩；
- 二阶矩；
- step 计数等。

这些额外状态就是 optimizer state。

12. Optimizer state：优化器状态

Optimizer state 是优化器为了更新参数而保存的额外数据。

以 Adam 为例，每个参数通常会对应：

- 参数本身；
- gradient；
- 一阶矩；
- 二阶矩。

所以 optimizer state 的规模可能非常大，甚至比参数本身还大。

在大模型训练中，optimizer state 经常是显存占用的大头之一。

因此，把 optimizer state 卸载到 CPU 或 SSD 是分层训练系统的重要策略。

13. Optimizer state shard：优化器状态分片

Optimizer state shard 是优化器状态的分片。

因为 optimizer state 很大，所以不能整体读写，而要切成多个 shard 或 tile。

在 optimizer step 阶段，系统可以按块处理：

```
读取一个 optimizer state tile  
用 gradient 更新当前 tile  
写回更新后的 tile  
读取下一个 tile
```

这样可以减少一次性内存占用，并且方便与 SSD I/O 重叠。

14. Optimizer step：优化器更新阶段

Optimizer step 是训练中真正更新参数的阶段。

完整训练 step 通常可以概括为：

forward → loss → backward → optimizer step

其中：

- forward 计算预测；
- loss 衡量预测错误；
- backward 计算梯度；
- optimizer step 更新参数。

optimizer step 通常需要访问：

- parameter；
- gradient；
- optimizer state。

如果 optimizer state 被卸载到 SSD，那么 optimizer step 就会涉及 SSD 的大量读写。所以本节特别强调 optimizer state 的 tile 化读写和流水写回。

15. Checkpoint：训练检查点

Checkpoint 是训练过程中的恢复点。

它通常保存：

- 模型参数；
- 优化器状态；
- 学习率调度器状态；
- 随机数状态；
- 当前训练 step；
- 其他恢复训练所需的信息。

为什么需要 checkpoint？

因为大模型训练时间很长，可能运行数天甚至数周。
如果中间机器故障，没有 checkpoint，就要从头训练。
有 checkpoint，就可以从最近保存点继续训练。

checkpoint 的特点是：

- 数据量大；
- 写入频率相对低；
- 可靠性要求高；
- 通常不应该阻塞每个训练 step。

因此，它适合采用后台顺序写入和版本管理。

16. Log / Profile：日志和性能分析数据

Log 是训练日志，例如 loss、学习率、吞吐、错误信息等。

Profile 是性能分析数据，例如某一步花了多少时间，GPU 是否空闲，I/O 是否阻塞等。

这类数据对调试和分析有用，但通常不是训练数学正确性必须的数据。

因此可以采用：

- 缓冲写入；
- 采样记录；
- 低优先级追加；
- 异常时提高记录频率。

它们不应该和 parameter、optimizer state 等关键路径数据抢 SSD 带宽。

17. Dataset cache：数据集缓存

Dataset cache 是训练数据的缓存。

例如文本数据经过分词、图片经过预处理后，可以缓存到 SSD 或本地磁盘中，避免每次重复处理。

dataset cache 的特点是：

- 读多写少；
- 服务于 data loader；
- 会间接影响训练速度。

它应该和写密集的 optimizer state、activation spill 等数据分开管理，避免读写互相干扰。

4. Offload：卸载

Offload 指的是把原本放在 GPU 显存中的数据转移到 CPU 内存或 SSD 中。

例如：

- GPU 放不下 optimizer state
- 把 optimizer state 放到 CPU 或 SSD
- 需要时再读回来

offload 的好处是可以训练更大的模型。

代价是增加了数据搬运和 I/O 调度复杂度。

5. Prefetch：预取

Prefetch 是提前读取未来会用到的数据。

例如当前 GPU 正在计算第 5 层，系统知道马上要计算第 6 层，于是提前把第 6 层参数从 SSD 读到 CPU 或 GPU。

这样当第 6 层真的开始计算时，数据已经准备好，GPU 不用等待。

预取的关键是时机：

- 预取太晚，GPU 会等数据；
- 预取太早，占用缓存空间，可能挤掉更重要的数据。

6. Cache：缓存

Cache 是把近期可能会再次使用的数据保存在更快的存储层中。

例如：

- GPU HBM 是最快缓存；
- CPU 内存是中间缓存；
- SSD 是大容量后端存储。

如果某个 parameter shard 很快还要用，就可以先留在 CPU 或 GPU 中，不必马上写回 SSD 或释放。

7. Evict：驱逐

Evict 是当缓存空间不足时，把暂时不用的数据移出当前层。

例如 CPU staging buffer 满了，就需要把一些短期不会用的数据驱逐到 SSD，给即将使用的数据腾出空间。

驱逐策略需要考虑：

- 哪些数据马上要用；
- 哪些数据很久以后才用；
- 哪些数据可以重算；
- 哪些数据必须保留。

8. Writeback：写回

Writeback 是把修改过的数据从高速层写回低速层。

例如 optimizer state 在 CPU 中被更新后，需要写回 SSD，以便后续继续训练或恢复。

写回可以立即做，也可以延迟做。

延迟写回可以合并多次修改，减少 SSD 写入压力。

9. Dirty：脏数据

Dirty 表示某个数据已经被修改，但还没有写回到 SSD。

例如：

optimizer state shard 从 SSD 读到 CPU

在 CPU 中被更新

此时 CPU 中版本比 SSD 中新

这个 shard 就是 dirty

dirty writeback 就是只把这些被修改过的数据写回，而不是所有数据都重新写一遍。

10. Critical path：训练关键路径

Critical path 指的是会直接影响训练 step 完成时间的路径。

例如：

forward 必须等参数到达

backward 必须等 activation 可用

optimizer step 必须等 optimizer state 可用

这些都是关键路径。

如果 SSD I/O 出现在关键路径上，GPU 就可能等待。

所以本节强调：

checkpoint、log 等后台 I/O 不应该阻塞 parameter shard 和 optimizer state 等关键路径 I/O。

11. I/O overlap：计算与 I/O 重叠

I/O overlap 指的是把数据读写和 GPU 计算同时进行。

例如：

GPU 正在计算第 5 层

SSD 正在读取第 6 层参数

CPU 正在准备第 7 层参数

这样 SSD I/O 的时间就被隐藏在 GPU 计算时间中。

如果做得好，GPU 不会明显感受到 SSD 的慢。

如果做得不好，GPU 就会频繁等待数据。

12. Lifecycle：生命周期

Lifecycle 指的是一个数据从产生、使用到释放的过程。

不同训练数据生命周期差异很大：

- activation 生命周期短；
- gradient 生命周期短；
- parameter 生命周期长；
- optimizer state 生命周期长；
- checkpoint 生命周期中长期；
- log/profile 生命周期取决于调试需求。

生命周期不同的数据不应该混在一起写 SSD。

短生命周期数据应该集中放置，方便快速复用和回收。

长期数据应该稳定保存，方便恢复。

13. Reuse distance：重用距离

Reuse distance 指的是一个数据从当前时刻到下一次被使用之间的间隔。

例如：

- 马上要用的数据，重用距离短；
- 很久以后才用的数据，重用距离长；
- 不再使用的数据，可以释放。

重用距离对缓存和预取非常重要。

如果一个 parameter shard 很快又要用，就应该尽量保存在 CPU 或 GPU 中。

如果它很久以后才会用，就可以驱逐到 SSD。

三、本节涉及的优化策略名词解释

1. Shard、Bucket、Tile、Slot 的区别

这几个词都表示“把数据组织成块”，但含义略有不同。

名词	含义	常见用途
Shard	分片，通常是长期稳定的数据切分	parameter shard、optimizer state shard
Bucket	桶，把多个小对象合并成一个较大 I/O 单元	参数预取、梯度通信、小 Tensor 合并
Tile	计算或更新时使用的小块	optimizer state tile
Slot	可复用的位置，常用于环形缓冲区	activation spill、临时数据

可以这样理解：

- **shard** 更像“切分后的长期数据块”；
- **bucket** 更像“为了批量处理临时合并的一组数据”；
- **tile** 更像“为了流水计算切出来的一块”；
- **slot** 更像“固定位置，使用完可以复用”。

2. 大块化 I/O

大块化 I/O 指的是把多个小 Tensor 合并成较大的 I/O 请求。

它解决的问题是：

小 Tensor 太多 → 小 I/O 太多 → SSD 和文件系统开销大

优化后变成：

多个小 Tensor → 合并成 shard/bucket/tile → 一次大块读写

好处是：

- 减少请求数量；

- 降低元数据开销；
- 更适合 SSD 顺序读写；
- 更容易和训练调度结合。

3. 对齐 I/O

对齐 I/O 是指 I/O 请求的起始位置和大小尽量符合存储系统更喜欢的边界。

简单理解：

SSD 和文件系统更喜欢规则、整齐的大块请求，而不是奇怪大小、随机位置的小请求。

对齐 I/O 可以减少额外拆分和管理开销。

4. Fixed Shard

Fixed Shard 是指为每个 shard 分配固定位置。

例如：

```
optimizer shard 0 → SSD 区域 A  
optimizer shard 1 → SSD 区域 B  
optimizer shard 2 → SSD 区域 C
```

好处是：

- 索引简单；
- 容易恢复；
- 适合 parameter 和 optimizer state。

风险是：

- 如果频繁小范围更新，可能变成随机覆盖写；
- 需要管理版本和偏移。

所以实际系统中一般会配合大块读写和批量写回。

5. Ring Buffer

Ring Buffer 是环形缓冲区。

可以把它想象成一圈固定位置：

slot 0 → slot 1 → slot 2 → slot 3 → 再回到 slot 0

当 slot 0 的数据已经不再需要，就可以被新的数据覆盖。

它适合 activation spill 这种短生命周期数据，因为 activation 用完后就可以释放。

好处是：

- 避免频繁创建和删除文件；
- 空间复用简单；
- 适合临时数据。

风险是：

- 必须准确判断 slot 中的数据是否已经不再需要；
- 如果提前覆盖，会破坏 backward 计算。

6. Append-only

Append-only 是只追加写，不覆盖旧数据。

例如 checkpoint：

checkpoint 1 写在后面
checkpoint 2 再追加到后面
checkpoint 3 继续追加

好处是：

- 写入顺序性好；
- 实现清晰；
- 适合 checkpoint 和日志。

代价是：

- 旧版本需要清理；
- 需要版本管理。

7. Log-structured

Log-structured 是把频繁更新的数据转换成日志追加。

例如 optimizer state 不直接覆盖旧位置，而是写一条新版本记录。
后续后台再把有效数据整理出来。

它的好处是顺序写友好。

但代价是需要后台整理，也就是 compaction。

在本节中，它属于增强探索策略，不一定是基础方案。

8. Recomputation

Recomputation 是“重算”。

它的思想是：

与其保存所有 activation，不如只保存一部分关键点，反向传播时重新计算中间 activation。

这样可以减少 activation 的保存和写入，但会增加 GPU 计算量。

这是一种典型的“用计算换存储”的方法。

9. Activation checkpointing

Activation checkpointing 是一种 activation 重算策略。

它不是普通 checkpoint。

这个名字容易混淆。

区别如下：

名词	含义
Checkpoint	保存训练恢复点，用于故障后继续训练
Activation checkpointing	只保存部分 activation，backward 时重算其余 activation

所以：

- checkpoint 是为了容错恢复；
- activation checkpointing 是为了省显存和减少 activation 保存。

10. CPU Cache

CPU Cache 在这里指用 CPU 内存缓存一部分训练状态。

例如 optimizer state 从 SSD 读到 CPU 后，如果很快还要用，就先不写回 SSD，而是留在 CPU 中。

这样可以减少 SSD 读写次数。

代价是占用 CPU 内存。

11. Dirty Writeback

Dirty Writeback 是脏数据写回。

只有被修改过的数据才需要写回 SSD。

没有修改的数据不用重复写。

例如 optimizer state shard 被更新后标记为 dirty。

之后系统在合适时间批量写回。

好处是：

- 减少重复写入；
- 方便合并写入；
- 减轻 SSD 压力。

12. Version Management

Version Management 是版本管理。

主要用于 checkpoint。

训练中可能保存很多 checkpoint，如果都保留，会占用大量 SSD 空间，也会产生大量写入。

版本管理可以设置策略：

- 保留最近几个 checkpoint

- 保留关键里程碑 checkpoint

- 删除过旧 checkpoint

这样既保留恢复能力，又避免无限制写入。

13. Sampling

Sampling 是采样记录。

主要用于 log/profile。

不是每一步、每个事件都完整写入，而是按一定间隔或条件记录。

例如：

- 每 100 step 记录一次详细 profile

- 平时只记录 loss、吞吐等摘要

- 异常时提高日志频率

这样可以减少低价值写入。

14. Write Coalescing

Write Coalescing 是写入合并。

它和大块化 I/O 很接近，强调把多个小写请求合并成一个大写请求。

例如：

10 个小 Tensor 分别写 SSD

改成：

先放入内存 buffer，再一次性写入 SSD

这样可以减少小随机写和元数据开销。

四、用一轮训练过程理解 3.4 节

为了更直观，可以把一次训练 step 理解成下面几个阶段。

阶段 1：Forward

读取参数 → 计算输出 → 产生 activation

此时系统需要：

- 把当前 layer 的 parameter 准备好；
- 可能提前预取后续 layer 的 parameter；
- 决定 activation 放 GPU、CPU，还是后续重算。

SSD 优化重点：

提前预取 parameter shard

避免 GPU 等待 SSD

activation 尽量不直接大量写 SSD

阶段 2：Backward

读取 activation → 计算 gradient

此时系统需要：

- 使用 forward 阶段产生的 activation；
- 计算 parameter 对应的 gradient；
- 释放不再需要的 activation；
- 必要时读取被卸载的数据。

SSD 优化重点：

提前准备 backward 需要的数据

短生命周期数据用 ring buffer 或重算处理

gradient 尽量不长期落盘

阶段 3：Optimizer Step

读取 optimizer state → 用 gradient 更新 parameter → 写回 optimizer state

这是 SSD 卸载训练中非常重要的阶段。

因为 optimizer state 很大，如果在 SSD 上，就需要大量读写。

SSD 优化重点：

optimizer state 按 tile 读取

当前 tile 计算时预取下一 tile

上一 tile 更新完成后异步写回

形成流水

也就是：

读下一块 + 算当前块 + 写上一块

阶段 4：Checkpoint / Log / Profile

- 保存恢复点
- 写训练日志
- 记录性能数据

这些通常不是每个训练 step 的最核心计算。

SSD 优化重点：

- 后台写入
- 低优先级
- 顺序追加
- 不要阻塞 `parameter` 和 `optimizer state` 的关键 I/O

五、3.4 节讲稿

下面这部分可以直接作为你讲解 3.4 节时的讲稿使用。

开场讲稿

这一节介绍的是面向 SSD 特性的训练 I/O 优化方法。

在大模型训练中，GPU 显存通常无法容纳全部训练数据。除了模型参数本身，训练过程还需要保存梯度、优化器状态、activation、checkpoint、日志和 profile 数据。其中优化器状态和 activation 的规模都可能非常大。如果所有数据都放在 GPU 显存中，显存很快就会成为瓶颈。

因此，我们引入 GPU、CPU 和 SSD 组成的分层存储结构。GPU HBM 速度最快，但容量有限；CPU 内存容量更大，但速度低于 GPU 显存；SSD 容量最大，成本较低，但访问延迟更高，并且不适合大量小粒度随机写。也就是说，SSD 可以扩展训练可用存储空间，但如果直接把 Tensor 一个一个写到 SSD 上，反而会产生大量小 I/O 和随机 I/O，导致训练吞吐下降。

所以这一节的核心问题是：如何让训练系统以更适合 SSD 的方式读写数据。我们不是简单地把数据放到 SSD，而是根据训练数据对象的生命周期、访问模式和可靠性要求，对数据进行组织、预取、缓存、

驱逐和写回。

本节主要包括四类方法：第一，大块化与对齐 I/O；第二，顺序化写入与生命周期感知放置；第三，分层缓存、异步预取与计算 I/O 重叠；第四，写入削减与 SSD 寿命优化。

训练数据对象讲解稿

在介绍具体优化方法之前，首先需要区分训练过程中不同的数据对象。

第一类是 parameter shard，也就是参数分片。模型参数是训练过程中长期存在的数据，forward 和 backward 都需要使用它。由于模型很大，参数通常会被切分成多个 shard。系统需要根据 layer 的执行顺序，在合适的时间把需要的参数分片预取到 CPU 或 GPU 中。如果参数没有及时到达，GPU 就会等待。

第二类是 gradient shard，也就是梯度分片。梯度是在 backward 阶段产生的，用于 optimizer step 更新参数。梯度的生命周期比较短，一般在更新完成后就可以释放。因此，gradient 不适合作为长期 SSD 数据，只有在内存压力很大时才考虑临时 spill。

第三类是 optimizer state shard，也就是优化器状态分片。以 Adam 优化器为例，除了参数和梯度，还需要维护一阶矩、二阶矩等状态。这些状态的数据量很大，是显存占用的重要来源。因此，optimizer state 是 SSD 卸载中的重点对象。在 optimizer step 阶段，系统需要读取 optimizer state，完成更新后再写回。

第四类是 activation spill。Activation 是 forward 阶段产生的中间结果，backward 阶段计算梯度时需要用到。Activation 的生命周期较短，但数据量可能很大。系统通常优先采用 activation checkpointing 和 recomputation，也就是只保存部分关键 activation，其他 activation 在 backward 时重新计算。只有当 CPU 内存不足或重算代价较高时，才把部分 activation 临时写入 SSD。

第五类是 checkpoint。Checkpoint 是训练恢复点，用于训练中断后继续恢复。它通常包含模型参数、优化器状态、训练步数、随机数状态等。Checkpoint 数据量大，但写入频率较低，一般可以作为后台顺序写入任务处理。

第六类是 log 和 profile。它们主要用于训练监控和性能分析，例如记录 loss、吞吐、GPU 利用率和 I/O 等待情况。这类数据不是训练数学正确性必须的数据，因此可以采样、缓冲和低优先级写入。

第七类是 dataset cache。它是数据加载过程中使用的缓存，通常读多写少。它应该与写密集的训练卸载数据分开管理，避免互相干扰。

通过这样的分类，系统可以为不同数据对象设计不同的 SSD I/O 策略，而不是把所有数据都当成普通文件处理。

六、3.4.1 大块化与对齐 I/O 方法讲稿

接下来介绍第一种方法：大块化与对齐 I/O。

这个方法主要解决的问题是小 I/O 过多。

训练过程中有大量 Tensor，尺寸也不一致。如果系统按照每个 Tensor 单独向 SSD 读写，就会产生大量小请求。这些请求本身的数据量可能不大，但每个请求都需要提交、调度、维护元数据，还可能导致 SSD 内部进行复杂的地址管理。请求数量过多时，SSD 的有效吞吐会下降，训练系统也会产生额外等待。

因此，我们希望把零散的小 Tensor 合并成更大的 I/O 单元。这里可以使用 shard、bucket、tile 和 slot 等组织方式。

对于 parameter shard，可以按照 layer 或 rank 合并成预取 bucket。因为模型的计算顺序通常是按 layer 进行的，所以系统可以提前知道后续 layer 会用到哪些参数，并把这些参数合并读取。

对于 optimizer state，可以按照 shard 或 tile 进行组织。Optimizer step 不一定要一次性处理全部优化器状态，而是可以一块一块处理。系统读取当前 tile，完成更新，然后写回，同时预取下一个 tile。

对于 activation spill，可以按照 ring buffer slot 组织。Activation 生命周期短，用完即可释放，因此可以使用固定 slot 循环复用。

对于 checkpoint，可以按照 rank 和 shard 组织成较大的分片文件，而不是写成大量小文件。

对于 log 和 profile，可以先在内存中按时间窗口缓冲，再统一追加写入。

这里需要注意，大块化不是越大越好。大块化的关键是 I/O 单元要和训练调度单元匹配。如果只是盲目把很多数据合并到一起，可能会把当前不需要的数据也读进来，占用 CPU 缓冲区，造成读放大和缓存浪费。

例如，当前马上要计算第 5 层，如果我们为了形成大块 I/O，把第 5 层和很久之后才会用到的第 20 层参数一起读取，那么虽然 I/O 请求变大了，但第 20 层参数会提前占用缓存空间，可能挤掉真正急需的数据。

因此，大块化应该服务于训练访问顺序。Parameter 的合并要服务于 layer 执行顺序，optimizer state 的合并要服务于 tile 更新顺序，activation 的组织要服务于 backward 重用顺序。

为了实现这一点，系统需要维护卸载对象元数据，包括对象类型、layer 编号、shard 编号、数据大小、生命周期、访问模式、重用距离等信息。调度器根据这些元数据决定哪些数据可以合并，哪些数据需要提前读，哪些数据可以延迟写回。

所以，大块化与对齐 I/O 不只是一个存储层优化，它实际上是训练运行时的数据组织方法。

七、3.4.2 顺序化写入与生命周期感知放置方法讲稿

第二种方法是顺序化写入与生命周期感知放置。

SSD 相比机械硬盘有更好的随机访问能力，但这并不意味着随机写完全没有代价。SSD 更喜欢连续、大块、生命周期相近的数据流。如果训练系统把短生命周期 activation、长期 checkpoint、频繁更新的 optimizer state 和低优先级日志混在一起写，就会增加 SSD 内部空间管理和数据搬移压力。

因此，这里的优化有两个层次。

第一，是顺序化写入。

也就是说，尽量让数据以连续追加或大块写入的方式进入 SSD，而不是大量随机覆盖。

第二，是生命周期感知放置。

也就是说，把生命周期相似的数据放在一起，把生命周期差异大的数据分开。

具体来说，parameter shard 和 optimizer state shard 是长期训练状态，需要稳定索引。它们适合使用 fixed shard，也就是为每个 shard 分配稳定位置，方便后续读取和恢复。

Activation spill 是短生命周期临时数据，适合 ring buffer。系统可以预先分配一块连续区域，划分成多个 slot。Forward 阶段产生的 activation 写入某个 slot，等 backward 使用完成后，这个 slot 就可以被复用。这样可以避免频繁创建和删除小文件。

Checkpoint 是中长期可靠性数据，适合 append-only 或分片顺序写。每次保存 checkpoint 时，可以生成新的版本，而不是到处随机覆盖旧文件。旧版本由后台版本管理机制清理。

Log 和 profile 是诊断数据，适合缓冲追加。正常训练时只记录摘要信息，异常时再提高记录频率。

Dataset cache 是读密集数据，应与写密集卸载数据分离。否则，写密集的 optimizer state 或 activation spill 可能干扰数据加载路径。

这一小节的核心思想是：

不同训练数据的生命周期不同，所以它们不应该混放。短生命周期数据集中放置，方便快速复用；长期数据稳定放置，方便恢复；低优先级数据后台追加，避免阻塞训练关键路径。

如果 SSD 支持主机侧放置提示或类似接口，训练框架还可以把这些语义传递给 SSD。例如告诉 SSD：这部分是短生命周期 activation，那部分是长期 checkpoint，另一部分是频繁更新 optimizer state。这样 SSD 可以更好地组织内部空间。

八、3.4.3 分层缓存、异步预取与计算 I/O 重叠方法讲稿

第三种方法是分层缓存、异步预取与计算 I/O 重叠。

这一节解决的问题是：SSD 比 GPU 慢，那么如何避免 GPU 等 SSD？

在训练中，最昂贵的资源通常是 GPU。如果 GPU 因为等待参数、activation 或 optimizer state 从 SSD 读出来而空闲，那么整体训练效率就会下降。因此，我们希望 SSD I/O 尽量隐藏在 GPU 计算过程中。

系统采用三级结构：

GPU HBM
CPU 内存 / Staging Buffer
SSD

GPU HBM 保存当前计算立即需要的数据。

CPU 内存作为中间缓存，负责承接 SSD 和 GPU 之间的数据搬运。

SSD 保存暂时放不进 GPU 或 CPU 的大规模训练状态。

在 forward 阶段，系统可以根据 layer 顺序提前预取后续 layer 的参数。比如 GPU 正在计算第 5 层时，系统可以异步读取第 6 层或第 7 层参数。这样等 GPU 真的要计算第 6 层时，参数已经准备好。

在 backward 阶段，系统需要根据反向传播顺序准备 activation 和参数。因为 backward 通常和 forward 顺序相反，所以系统可以预测接下来需要哪些 activation。已经完成 backward 使用的 activation 可以释放，或者标记为可复用。

在 optimizer step 阶段，系统重点处理 optimizer state。Optimizer state 很大，如果一次性全部读取和写回，会造成明显阻塞。因此可以按 tile 流水处理。

流水方式可以理解为：

```
读取下一块 optimizer state
计算当前块
写回上一块
```

这样读、算、写可以同时推进，而不是串行等待。

为了实现这种重叠，系统需要异步 I/O 队列和 CPU 缓冲池。异步 I/O 队列负责同时向 SSD 提交多个请求；CPU 缓冲池负责暂存从 SSD 读取的数据，或者暂存准备写回 SSD 的数据。

调度器需要决定几个事情：

- 哪些数据要提前预取；
- 哪些数据要保留在 CPU 或 GPU；
- 哪些数据可以驱逐到 SSD；
- 哪些写入可以延迟；
- 哪些后台任务要降级。

例如，如果 checkpoint 写入正在占用 SSD，而此时 optimizer state 需要读取，那么 checkpoint 应该让路，因为 optimizer state 位于训练关键路径，而 checkpoint 通常是后台任务。

这一小节的核心思想是：

SSD I/O 不一定要完全消失，但要尽可能不出现在 GPU 等待路径上。

只要 I/O 能够和计算重叠，训练吞吐就可以保持稳定。

九、3.4.4 写入削减与 SSD 寿命优化方法讲稿

第四种方法是写入削减与 SSD 寿命优化。

前面几种方法主要解决“怎么更高效地读写”，这一节解决的是“哪些数据可以少写，甚至不写”。

大模型训练中，并不是所有数据都必须完整写入 SSD。不同数据的重要性不同。

Parameter 和 optimizer state 是训练状态的核心数据，必须保证正确性。

Activation 多数情况下可以重算。

Log 和 profile 可以采样。

Checkpoint 很重要，但可以控制保存频率和版本数量。

第一种写入削减方法是 recomputation，也就是重算 activation。

Activation 在 backward 中需要使用，但不一定全部保存。系统可以只保存少量关键 activation，其他中间结果在 backward 时重新计算。这样可以减少 activation 对 GPU、CPU 和 SSD 的占用。

第二种方法是 CPU cache 和 dirty writeback。

Optimizer state 每一步都会更新，如果每次更新都立即写回 SSD，会产生大量写入。系统可以先把近期可能使用的 optimizer shard 保存在 CPU 中，并标记哪些 shard 被修改过。等到合适时机，再把多个 dirty shard 合并写回 SSD。

第三种方法是 checkpoint 版本管理。

Checkpoint 用于故障恢复，不能随便删除，但也不需要无限制保留。系统可以保留最近几个 checkpoint 和关键里程碑 checkpoint，删除过旧版本。这样既保证恢复能力，又减少不必要写入。

第四种方法是日志和 profile 采样。

正常训练时不需要每一步都写详细 profile，可以只记录关键指标。出现异常时，再提高记录频率。这样可以减少低价值写入。

第五种方法是小 Tensor 写入合并。

对于确实需要写入的小对象，系统先在内存中合并，再统一写 SSD，而不是逐个写入。

这里必须强调一个原则：

写入削减不能破坏训练正确性。

Parameter、optimizer state、随机数状态、学习率调度器状态等关键数据必须在 checkpoint 中保持一致。

Activation 重算必须保证 backward 结果正确。

日志采样虽然可以降低粒度，但要保留必要的错误信息和诊断线索。

所以写入削减不是简单地少写，而是根据训练语义判断哪些数据可以安全减少保存。

十、最后总结讲稿

总结来说，3.4 节的核心是：在 GPU-CPU-SSD 分层协同训练中，SSD 不能被当作普通磁盘随意读写，而应该根据训练数据对象的特点进行专门优化。

大块化与对齐 I/O 解决的是小 Tensor 太多、小 I/O 太多的问题。它把零散数据组织成 shard、bucket、tile 和 slot，减少请求数量，提高 SSD 访问效率。

顺序化写入与生命周期感知放置解决的是随机写和冷热混放问题。它根据 parameter、optimizer state、activation、checkpoint、log/profile 等数据的生命周期，把不同类型数据分开组织。

分层缓存、异步预取与计算 I/O 重叠解决的是 GPU 等待 SSD 的问题。它通过 GPU、CPU 和 SSD 之间的协同调度，让数据提前到达，让 I/O 尽量隐藏在计算过程中。

写入削减与 SSD 寿命优化解决的是写入总量过大的问题。它通过 activation 重算、optimizer state 缓存写回、checkpoint 版本管理和日志采样，减少不必要写入。

这四类方法是互补的。

大块化让 I/O 更高效，顺序化让 SSD 更容易处理，重叠让 GPU 少等待，写入削减从源头减少 SSD 压力。最终目标是在保证训练正确性和故障恢复能力的前提下，提高训练系统的读写效率，降低 I/O 阻塞，并延长 SSD 的使用寿命。

十一、容易混淆的概念对照

1. Parameter 和 Optimizer State 的区别

Parameter 是模型要学习的权重。

Optimizer state 是优化器为了更新这些权重而维护的辅助状态。

例如 Adam 中：

parameter: 模型权重

optimizer state: 一阶矩、二阶矩等

parameter 是模型本体。

optimizer state 是训练过程中的辅助数据。

2. Activation 和 Gradient 的区别

Activation 是 forward 产生的中间结果。
Gradient 是 backward 产生的参数更新方向。

forward → activation
backward → gradient

activation 是为了计算 gradient。
gradient 是为了更新 parameter。

3. Checkpoint 和 Activation Checkpointing 的区别

这两个名字都带 checkpoint，但含义不同。

概念	作用
Checkpoint	保存训练恢复点，防止故障后从头训练
Activation checkpointing	少保存 activation，通过 backward 时重算节省显存

Checkpoint 是为了容错。
Activation checkpointing 是为了节省显存和减少 activation 保存。

4. Offload 和 Cache 的区别

Offload 是把数据从快存储转移到慢存储，例如从 GPU 放到 CPU 或 SSD。
Cache 是把可能再次使用的数据保存在较快位置，减少后续读取等待。

offload: 放不下，往下放
cache: 可能要用，先留着

5. Prefetch 和 Read 的区别

Read 是普通读取。

Prefetch 是提前读取。

read: 需要时再读

prefetch: 还没用，但预测马上要用，所以提前读

预取的目标是避免 GPU 等待。

6. Writeback 和 Checkpoint 的区别

Writeback 是把修改过的数据写回 SSD，例如 optimizer state 更新后写回。

Checkpoint 是保存一个完整或部分训练恢复点。

writeback 是训练状态日常维护。

checkpoint 是故障恢复机制。

十二、你可以用来理解整节的简化比喻

可以把大模型训练想象成一个大型工厂。

GPU 是工人，速度最快，但工作台很小。

CPU 内存是旁边的临时货架，容量大一些，但拿取速度慢一些。

SSD 是仓库，容量最大，但取货最慢。

训练数据就是各种零件：

- parameter 是机器本体零件；
- gradient 是调整零件的指令；
- optimizer state 是维修记录和历史状态；
- activation 是加工过程中的临时半成品；
- checkpoint 是整个工厂状态的备份；
- log/profile 是工作日志；

- dataset cache 是原材料缓存。

如果每个小零件都单独从仓库取，效率会很低。

所以要把零件打包成箱，也就是大块化 I/O。

如果短期零件、长期备份、日志文件混在一个仓库区域，管理会很乱。

所以要按生命周期分区放置。

如果工人等仓库取货，生产线就停了。

所以要提前预取，让仓库取货和工人加工同时进行。

如果有些半成品可以现场重新做，就不一定要全部放进仓库。

这就是 recomputation。

这就是 3.4 节的整体思想。